

new/delete vs malloc/free

C++ հարցազրույցի ամբողջական ուղեցույց

Այս ուղեցույցը մանրամասն համեմատում է `new / delete`-y (C++) և `malloc / free`-y (C)՝ constructor/destructor, type safety, sizeof, exception, orinaknerov ev interview harcerov.

1. Հիմնական տարբերությունը

```
// C++ (new/delete)
int* p = new int(10);
delete p;

// C (malloc/free)
int* q = (int*)malloc(sizeof(int));
*q = 10;
free(q);
```

Amenamec tarberutyuny. **new/delete-y kanchum e constructor/destructor**, malloc/free-y՝ ոչ.

2. Constructor / Destructor

new -> constructor kanchvum

```
class A {
public:
    A() { cout << "Constructor\n"; }
    ~A() { cout << "Destructor\n"; }
};

A* p = new A();           // Output: Constructor
delete p;                  // Output: Destructor
```

malloc -> constructor CHI kanchvum

```
A* q = (A*)malloc(sizeof(A)); // OCHMI Constructor -- miayn raw memory
free(q);                       // OCHMI Destructor
```

`malloc` -y miayn raw byte-er e vercnun -> object chi "kañucvum" (member-nery chen initialize): Class-eri hamar sa vtangavor e.

3. Type Safety

```
int* p = new int;           // veradarcnum e int* (type-safe)
int* q = (int*)malloc(4);   // veradarcnum e void* -> petq e cast
```

`new` -y veradarcnum e chisht type-y (`int*`): `malloc` -y veradarcnum e `void*`, petq e dzerqov cast anel -> aveli sxalapatek.

4. Size hashvark

```
int* p = new int[10];       // compiler-y ineqy gitn chapy
int* q = (int*)malloc(10 * sizeof(int)); // dzerqov sizeof
```

`new` -i hamar chapy avtomat e; `malloc` -i ham ar dzerqov `sizeof` -> hesht e sxalvel.

5. Exception vs nullptr

```
int* p = new int[1000000000]; // ANHAJOXUTYAN depqum -> throw std::bad_alloc
int* q = (int*)malloc(...);    // ANHAJOXUTYAN depqum -> veradarcnum nullptr
```

`new` -y anhajoxutyany depqum `throw` e anum (`std::bad_alloc`): `malloc` -y veradarcnum e `nullptr` (petq e dzerqov stugel).

6. Resize

```
// malloc uni realloc
int* q = (int*)malloc(10 * sizeof(int));
q = (int*)realloc(q, 20 * sizeof(int)); // chapy poxel

// new CHUNI realloc -- petq e nor new + copy + delete
```

`malloc` -y uni `realloc` (chapy poxel); `new` -i hamar chka` petq e nor block, copy, hin delete (kam ogtagorci vector).

7. Համեմատական աղյուսակ

	new/delete	malloc/free
Lezu	C++	C
Constructor/Destructor	ԱՅՈ	ՈՉ
Return type	chisht type (int*)	void* (cast petq)
Size hashvark	avtomat	dzerqov (sizeof)
Anhajoxutyun	throw bad_alloc	return nullptr
Resize	chka (nor+copy)	realloc
Operator/function	operator	function
Overridable	ayo (operator new)	voch

8. Karevor kanonner

- new -> delete | new[] -> delete[]
- malloc -> free
- MI XARNIR: new-y free-ov, kam malloc-y delete-ov -> undefined behavior

```
int* p = new int;
free(p);           // SXAL -- undefined behavior

int* q = (int*)malloc(4);
delete q;          // SXAL -- undefined behavior
```

9. Erb vory ogtagorcel

C++ kod	-> new/delete (kam aveli lav` smart pointers)
C kod / C API	-> malloc/free
Modern C++	-> OCHMI meky unix -> make_unique / make_shared / vector

10. Interview harcer & patasxanner

new vs malloc himnakan tarberutyun?

*new-y kanchum e constructor (u delete **destructor**); malloc/free nž. new-y type-safe, malloc-y void* return.*

new anhajoxutyan depqum inch e anum?

throw std::bad_alloc (malloc-y nullptr veradarcnum).

Kareli e new-i het free ogtagorcel?

Voch -> undefined behavior. new->delete, malloc->free.

Inchu new-y aveli lav e class-eri hamar?

Constructor-y kanchum -> object chisht initialize; malloc-y miayn raw memory (member-ner uninitialized).

delete vs delete[]?

new->delete, new[]->delete[]. Sxal zuygy -> UB.

Modern C++-um inchpes?

OCHMI raw new/delete/malloc -> make_unique/make_shared/vector (RAII).

Cheat Sheet

new/delete (C++):	malloc/free (C):
ctor/dtor kanchum	OCHMI ctor/dtor
type-safe (int*)	void* (cast petq)
size avtomat	sizeof dzerqov
throw bad_alloc	return nullptr
chka realloc	uni realloc
operator (overridable)	function

KANON:

new -> delete | new[] -> delete[] | malloc -> free
MI XARNIR (new+free / malloc+delete -> UB)

MODERN C++: ochmi meky -> make_unique / make_shared / vector